



Oviyam

Protée
IT, app design

Paris, on Wednesday 4 March 2020

Oviyam overview

Version	Date	Rédacteur	Commentaires
R0	4 March 2020	OG	First version

I) Introduction

Oviyam is a database in 4D, to create, store, manage all your graphical contents of your 4D databases resources folders.

You organize your Projects in Banks with Banks_Buttons and Banks_Sets, you can save in one click to one or more Banks_Paths.

It contains an object generator, with plenty parameters to generate Icons, 4D horizontal and vertical buttons.

It contains also a module for "4D toggle popup" export, for graphic material (btn_, icn_, icns_) and code generation.

All the system is based on ogTools component, offering a huge set of functionalities and changes with ease. It is based on Material Design (MD) indexed colors palette, with the extra added for transparent.



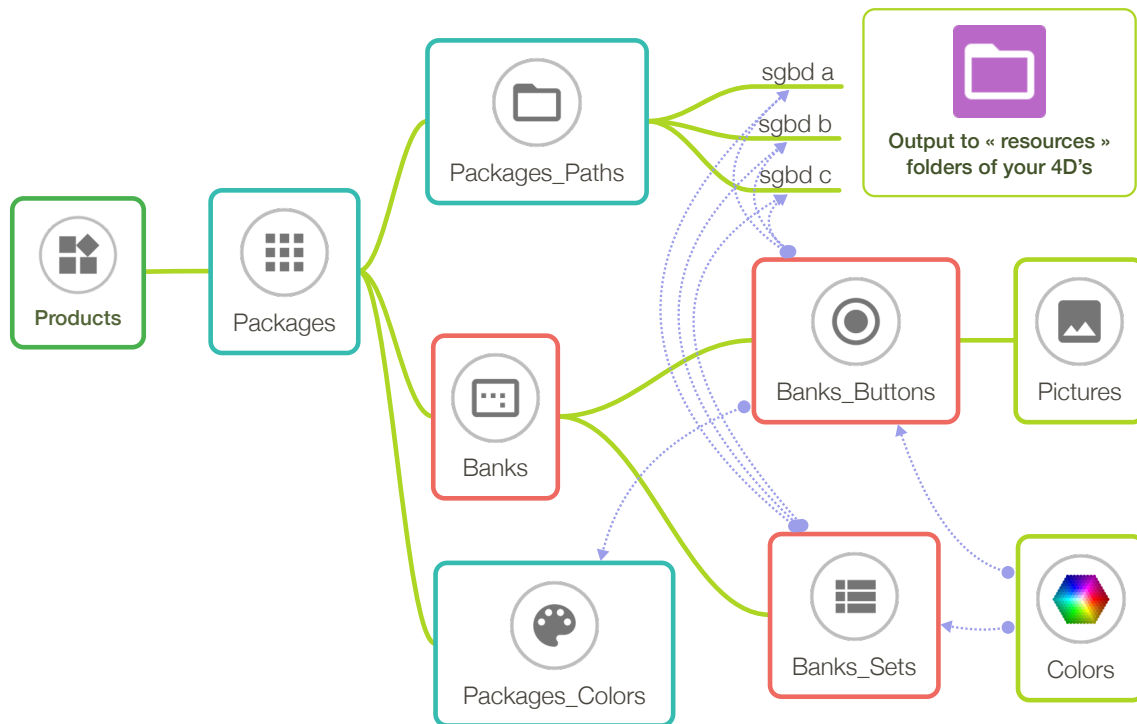
Protée sarl
au capital de 7 622 €
RC B 404 445 629

<http://www.protee.org>
info@protee.org
Tél : 0970 46 56 46
Mob : 0660 49 33 23

• 148 rue du Fbg St Denis
75010 PARIS
• 71 rue Adolphe Coll
31300 TOULOUSE

II) Architecture

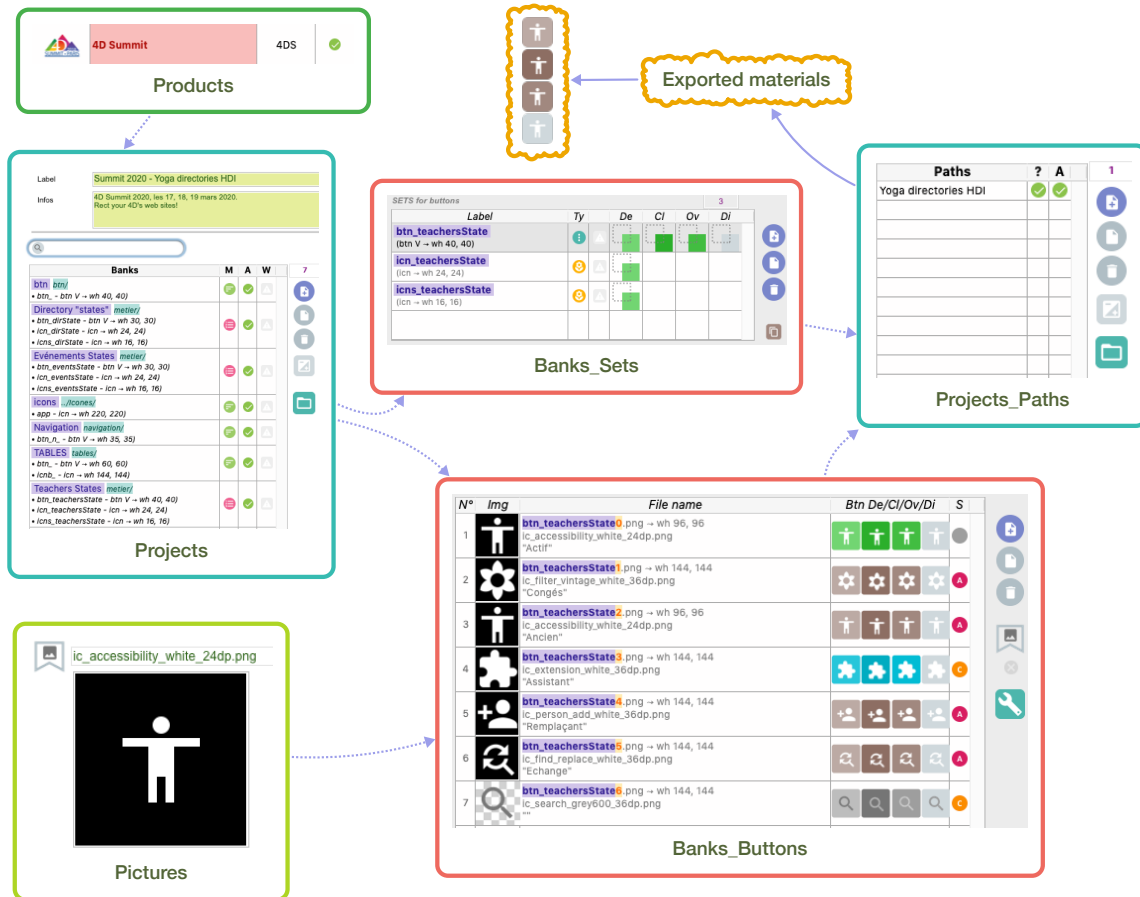
Here is an overview of the logical architecture.



- Products : regroups your Packages in products, and gives an icon to it.
- Packages : a group of Banks for your project. It contains Packages_Paths too.
- Packages_Paths : a list of destination folder where to export graphical contents. Usually a database has one destination, but for generic contents, you can have all your databases in destination.
- Packages_Colors : a choice of colors that override, when linked in Buttons, with enablers, the colors defined in Banks_Sets.
- Banks : it contains a list of sets (Banks_Sets) and a list of buttons (Banks_Buttons). Also a list of colors to use with your buttons is provided.
- Banks_Sets : all the parameters for an export : colors Default, Click, Over, Disabled, shape, type (icon, 4D H, 4D V), Picture, Text... Then you can export all Banks_Sets of a Bank in one click.
- Banks_Buttons : association between Banks and Pictures. It contains parameters too to override the regular parameters of a Banks_Sets.
- Pictures : here are the storage for images to use in your icons / buttons.
- Colors : a choice of colors to copy in destination Banks_Sets and Banks_Buttons. This table is not linked to anything.

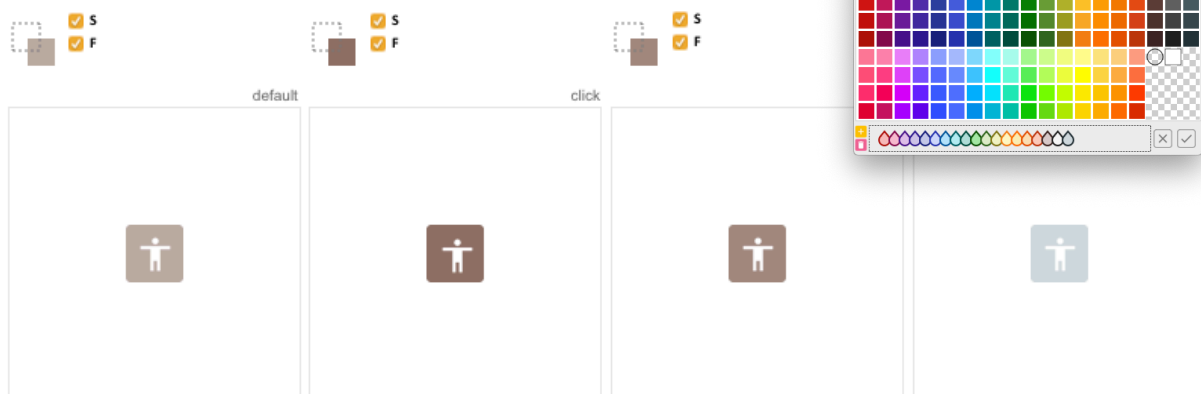
Demonstration

This is the flow of the database.



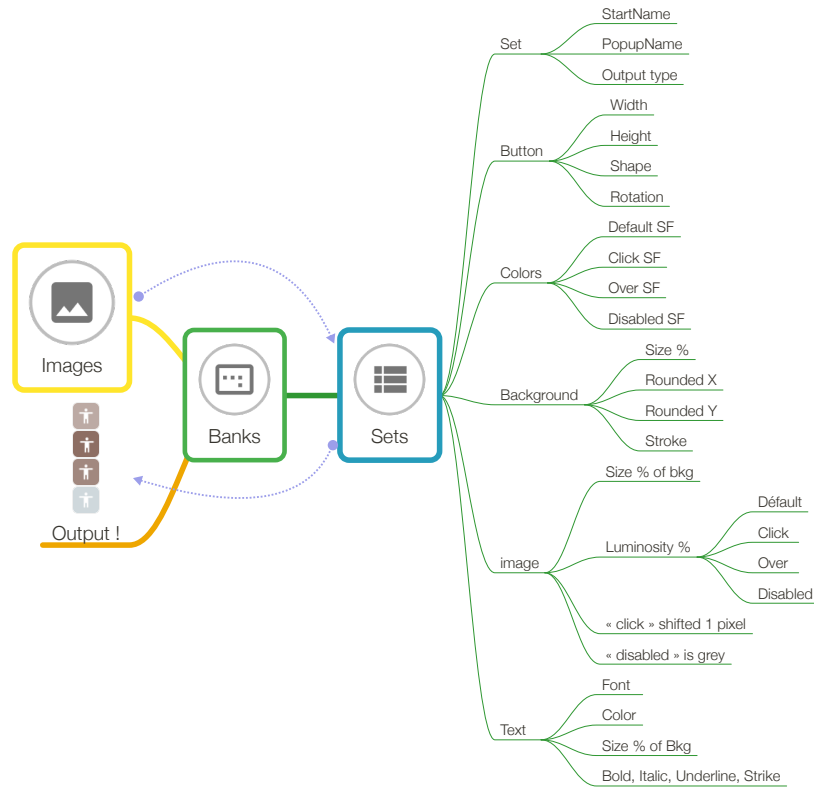
All the colors are MD indexed google palette, and Oviyam manages Default, Click, Over, Disabled, with overwrite enablers for each image and Stroke and Fill (S and F).

Overload colors & enablers



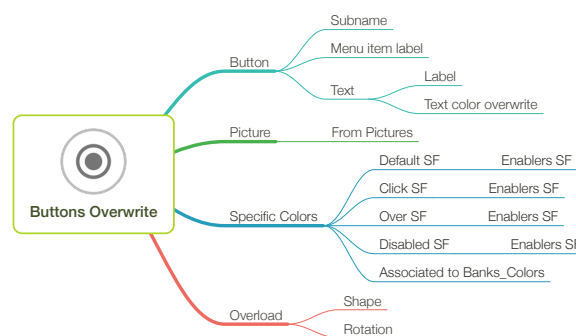
Banks_Sets parameters

Each bank is composed with Sets, where you define parameters to output Buttons depending on each Sets.



Banks_Buttons parameters

Each Button has parameters to overwrite the set's parameters locally.



Shapes

All pictures are generated with ogTools, the components dedicated to interfaces and UI. Here is the available shapes in ogTools → Oviyam :

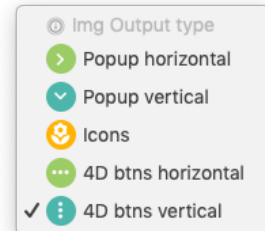




Output type

You define the output type in Banks_Sets.

- popup horizontal : all the buttons are calculated based on default, and are combined in one image horizontally, with startName.png.
- popup vertical : all the buttons are calculated based on default, and are combined in one image vertically, with startName.png.
- icons : all the buttons are calculated based on default, and are output one by one.
- 4D btns horizontal : all the buttons are calculated based on default, click, over, disabled, and are output one by one.
- 4D btns vertical : all the buttons are calculated based on default, click, over, disabled, and are output one by one.



Colors overwriting

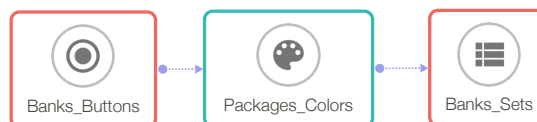
You define the colors in Banks_Sets as the lowest layer.

Then, in Banks_Buttons, you can overwrite colors in two more layers.

- with a link to a Packages_Colors, based on colors and enablers SF
- in Banks_Buttons itself with a local set of colors and enablers SF

The priority is from Banks_Buttons ,then Packages_Colors, then Banks_Sets.

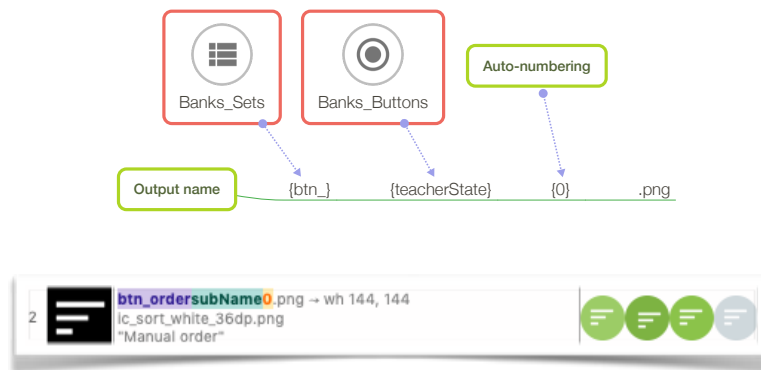
This allows you to manage exceptions in sets very efficiently.



Output names manager

All the output are .png files. You construct the names in that way :

- each Banks_Sets define the start-name of the output, like btn_, icn_, icns_, icnb_, ...
- each Banks_Buttons can give a sub-name added after the start-name.
- then, in auto-numbering mode, a count from "0" is added at the end, reset to "0" at each sub-name rupture - this allows to generate more than one group of counting items in one Bank.



Export manager

You can export your stuff from Projects, or from Banks.

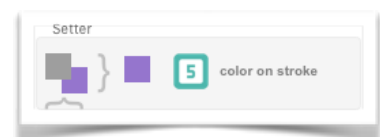
- from Projects : you can export all banks or the one selected. To all Paths or the one selected. All the Sets not in manual mode are exported.
- from Banks : you can export all Sets, or the one selected, of all buttons or the one selected. To all paths or the one selected in Packages.



A form opens to propose you this default behavior, you can then change locally.

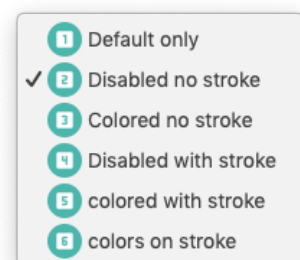
Setters (Mode + color → Strokes, Fills)

To set colors for Default, Click, Over and Disabled, each for stroke and fill is a little harassing. So a component exists to help you with some regular patterns. You select a mode, then you choose a color, and here it is !



This is available for Colors, Packages Colors, Banks Buttons.

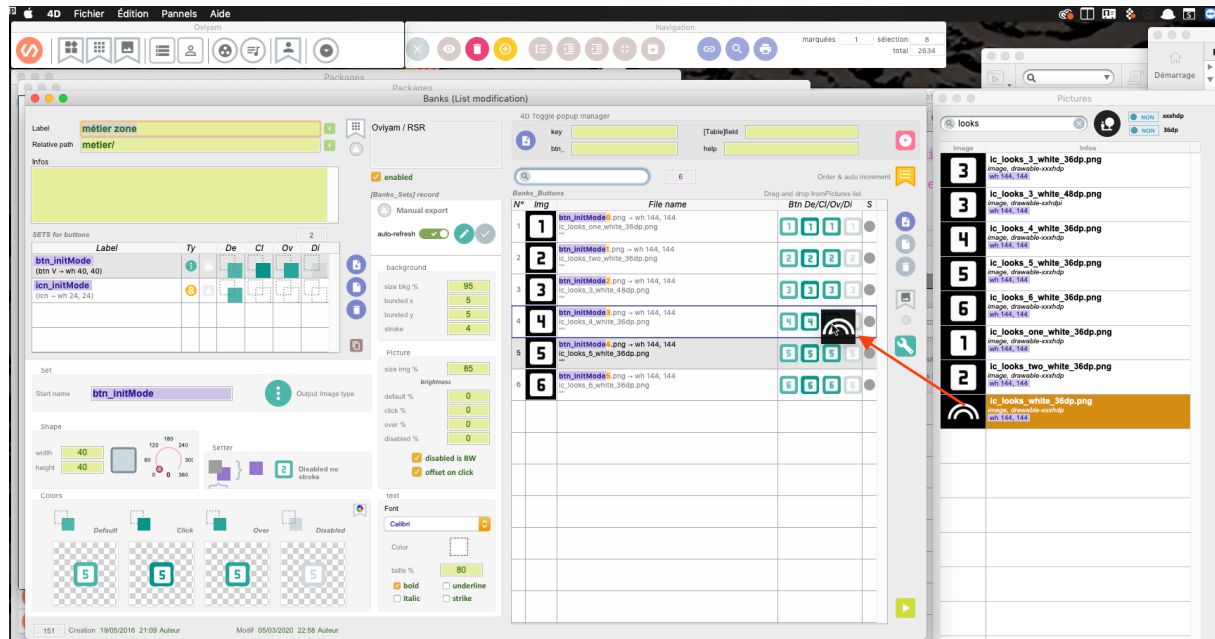
1. Default only
2. Disabled no stroke : stroke is transparent, and colors are for Fill only, with Disabled grey.
3. Colored no stroke : same as 2 with Disabled colored too.
4. Disabled with stroke : same as 2 with stroke for Click and Over.
5. Colored with stroke : same as 3 with Disabled colored too.
6. Colors on stroke : stuff on stroke. Default and Click fill white, Disabled grey.



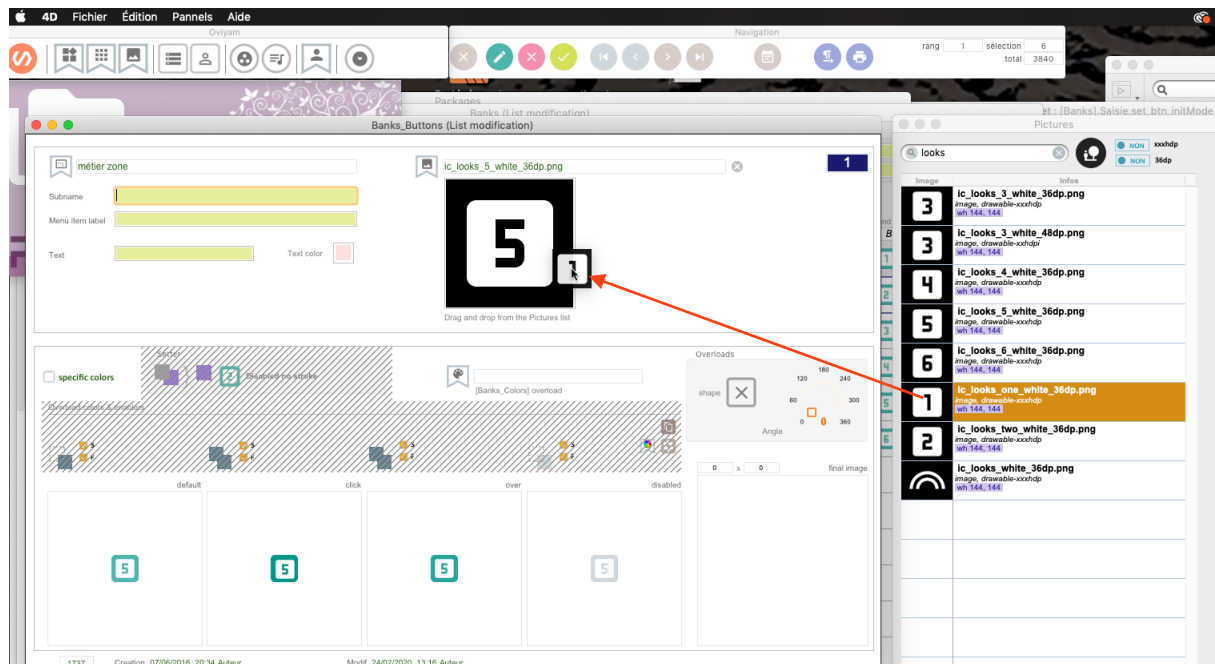
Drag drop Pictures

You can, from the Pictures list, drag and drop a record to:

- the Banks record on the Banks_Buttons sublist



- the Banks_Buttons record on the image to link

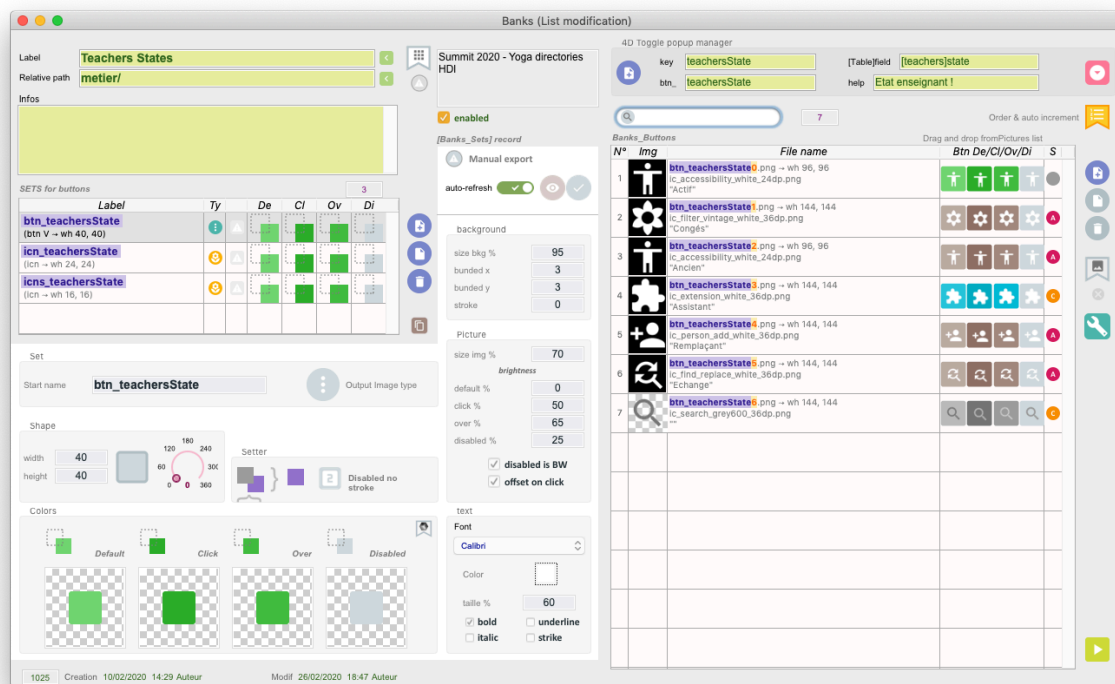


4D Toggle popup manager

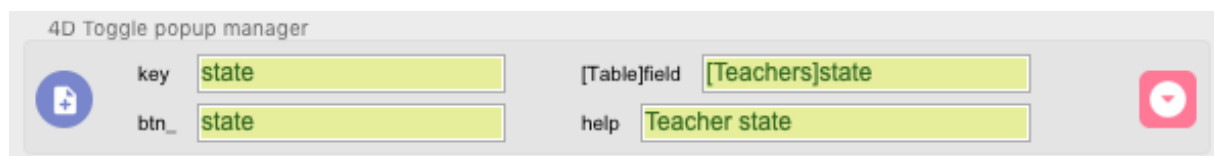
This is a Protée's module created to easily create a state button with a menu, and an optional idle state. This module is managed in a Bank.

Read carefully here "<https://www.protee.org/index.php/en/download-en/4d-free-bonus>" the document "4DSummit2018-Button in all its states EN r3".

Here is an example for a Teachers state popup.



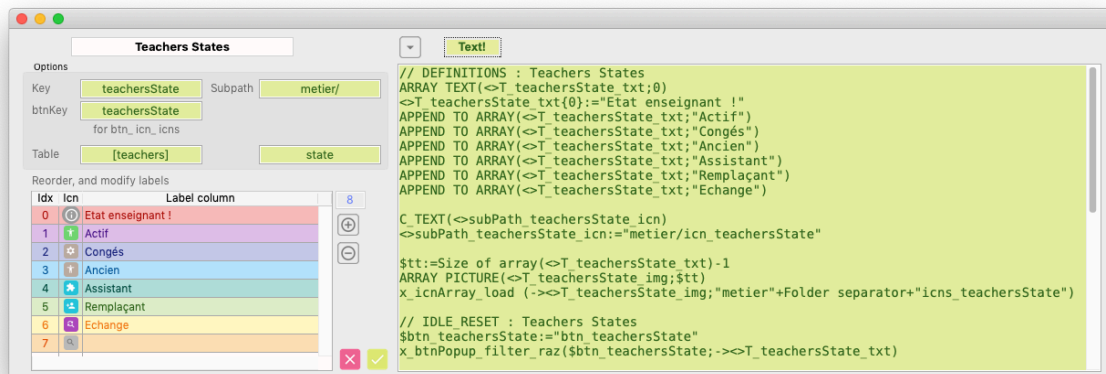
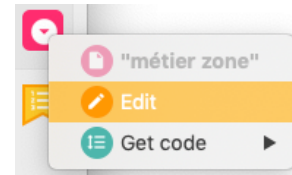
Banks zone for parameters



The blue button open a form where you can create btn_, icn_, icns_ Banks_Sets records in a snap.

- You create Banks_Buttons in the auto-numbering mode, and you name them like btn_state.
- In key, you give what is used to name the variable in the code generated.
- In btn_, you give the name used for btn_, icn_, icns_, here "state".
- In [Table]field, the field associated, here [Teachers]state.
- in Help, you give the help tip on the first menu item.

The pink button is a direct access to ogTools form and code generation.
Edit open the Form where you can easily change the "Menu item label"
for all the buttons.

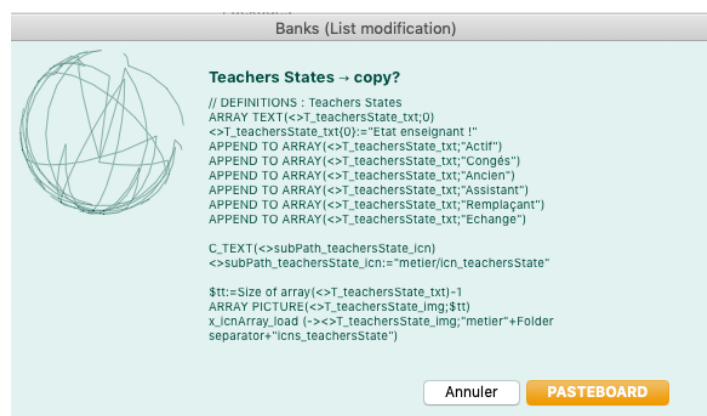
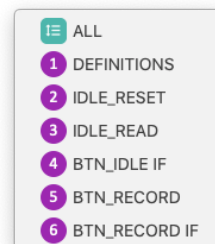


Get code gives you the code for initialization of what is needed to deal with the methods.

You will need: x_btnPopup_val, x_btnPopup_filter_init, x_btnPopup_filter. You will find it in ogTools HDI (free download).

Code generation:

- Definitions - all the code to execute at startup
- Idle reset - to reset an idle toggle popup to idle state
- Idle read - to read a toggle popup if active (not idle), and the value. For filter in listbox.
- Btn_idle if - to manage an idle toggle popup with if-endif to manage the change.
- Btn_record - to manage an idle toggle popup associated with a variable or a field.
- Btn_record if - same as Btn_record but with if-endif to manage the change.





You just have to paste the code where it has to be :)

And here is the result, for an idle popup and a field popup.

